

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2 - PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA12	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2 - PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	Deepika Shree. S	Reg. No.:	192521264
Department:	B.Tech IT	Date:	

ANNEXURE A

Pseudocode Writing Task (Algorithm Design) — Answers

Question 1

Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.

Aim:

To design an algorithm in which a DMA controller autonomously transfers a block of data from an I/O device to main memory, using cycle stealing to share the system bus, and notifying the CPU of completion through an interrupt so that the CPU is not required to poll or supervise the transfer.

Pseudocode:

```
BEGIN DMA_Transfer(device, memory_address, data_length)
    // Step 1: CPU initializes the DMA controller registers
    SET DMA.source_address      = device.port_address
    SET DMA.destination_address = memory_address
    SET DMA.transfer_length     = data_length
    SET DMA.mode                 = READ_FROM_DEVICE
    SET DMA.interrupt_on_completion = TRUE

    // Step 2: CPU starts DMA and is freed to do other work
    DMA.status = BUSY
    START DMA.controller
    CPU.continue_other_instructions()    // no CPU polling required

    // Step 3: DMA controller performs cycle-stealing transfer
    WHILE DMA.bytes_transferred < data_length DO
        REQUEST bus_control FROM CPU
        IF bus_control GRANTED THEN
            READ one_word FROM device.port_address
            WRITE one_word TO memory[DMA.destination_address]
            DMA.destination_address = DMA.destination_address + 1
            DMA.bytes_transferred   = DMA.bytes_transferred + 1
            RELEASE bus_control      // give bus back to CPU
        END IF
    END WHILE
```

```

// Step 4: Signal completion
DMA.status = COMPLETE
RAISE INTERRUPT(DMA_COMPLETE, device.id)
END DMA_Transfer

// Step 5: CPU's Interrupt Service Routine handles completion
ON INTERRUPT DMA_COMPLETE(device_id) DO
  SAVE CPU.context
  IF DMA.bytes_transferred == data_length THEN
    UPDATE process_control_block(device_id, "TRANSFER_DONE")
    CLEAR DMA.status
  ELSE
    LOG_ERROR("DMA transfer incomplete")
    RETRY_or_ABORT()
  END IF
  RESTORE CPU.context
  RETURN FROM INTERRUPT
END ON

```

Result:

The algorithm demonstrates that once the CPU programs the DMA controller with the source, destination, and length, it is completely free to execute other instructions while the DMA controller moves data word by word using cycle stealing (borrowing bus cycles only when the CPU is not using them). This minimizes CPU involvement to only two points: initiating the transfer and handling the completion interrupt raised in the ISR, drastically reducing CPU overhead compared to programmed I/O.

Question 2

Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.

Aim:

To design a vectored interrupt handling algorithm in which each device supplies its own vector address identifying its Interrupt Service Routine (ISR), and the interrupt controller queues, prioritizes, and dispatches multiple simultaneous requests in order of importance.

Pseudocode:

```

STRUCTURE InterruptRequest
  device_id
  vector_address
  priority_level      // lower number = higher priority
END STRUCTURE

DECLARE InterruptQueue[]           // holds pending IRQs
DECLARE InterruptVectorTable[0..N] // maps vector -> ISR address

BEGIN Interrupt_Controller_Cycle()
  LOOP FOREVER
    IF any_device.INT_line == ACTIVE THEN
      FOR EACH device THAT raised an interrupt DO
        irq = InterruptRequest(device.id, device.vector, device.priority)
        INSERT irq INTO InterruptQueue
      END FOR

      SORT InterruptQueue BY priority_level ASCENDING

      WHILE InterruptQueue NOT EMPTY DO
        current = REMOVE_HIGHEST_PRIORITY(InterruptQueue)

        IF current.priority_level <= CPU.current_priority_mask THEN
          CONTINUE // masked - wait till CPU priority drops
        END IF
      END WHILE
    END IF
  END LOOP
END

```

```

        END IF

        SEND acknowledge_signal TO current.device_id
        DISABLE interrupts WITH priority <= current.priority_level

        SAVE CPU.context (PC, registers, flags) ON stack
        ISR_address = InterruptVectorTable[current.vector_address]
        JUMP TO ISR_address
        EXECUTE Service_Routine(current.device_id)
        RESTORE CPU.context FROM stack

        ENABLE previously_disabled_interrupts
    END WHILE
END IF
END LOOP
END Interrupt_Controller_Cycle

PROCEDURE Service_Routine(device_id)
    SWITCH device_id
        CASE KEYBOARD:    handle_keyboard_input()
        CASE DISK:         handle_disk_transfer()
        CASE TIMER:        handle_timer_tick()
        CASE NETWORK:      handle_network_packet()
        DEFAULT:           LOG("Unknown device interrupt")
    END SWITCH
    CLEAR device.INT_line
END PROCEDURE

```

Result:

Every interrupting device places an interrupt request into a queue tagged with its vector address and priority level. The controller sorts the queue and always services the highest-priority request first, masking equal-or-lower priority interrupts during service to prevent priority inversion. Because each device carries its own vector, the CPU jumps directly to the correct ISR (keyboard, disk, timer, network, etc.) without polling every device, giving fast, priority-ordered, and device-specific dispatching.

Question 3

Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.

Aim:

To design a decision algorithm that dynamically chooses between memory-mapped I/O and I/O-mapped (isolated) I/O for a given device request, based on real-time measurement of required bandwidth, tolerable latency, and transfer size.

Pseudocode:

```

STRUCTURE IORequest
    device_id
    required_bandwidth    // MB/s
    max_tolerable_latency // microseconds
    data_size             // bytes
END STRUCTURE

CONSTANT BANDWIDTH_THRESHOLD = 100    // MB/s
CONSTANT LATENCY_THRESHOLD   = 10     // microseconds
CONSTANT LARGE_BLOCK_SIZE    = 4096   // bytes

BEGIN Select_IO_Technique(request)
    available_bandwidth = MEASURE_BUS_BANDWIDTH()
    current_latency     = MEASURE_IO_LATENCY()

    IF request.required_bandwidth >= BANDWIDTH_THRESHOLD

```

```

AND request.data_size > LARGE_BLOCK_SIZE THEN
    technique = MEMORY_MAPPED_IO
    // large transfer -> reuse full instruction set + DMA-friendly addressing

ELSE IF request.max_tolerable_latency <= LATENCY_THRESHOLD THEN
    technique = IO_MAPPED_IO
    // strict latency -> dedicated IN/OUT bus cycle is faster & predictable

ELSE
    // weigh both factors when neither condition dominates
    score_MMIO      = (available_bandwidth / request.required_bandwidth) * 0.6
    score_IOMapped   = (LATENCY_THRESHOLD / current_latency) * 0.4

    IF score_MMIO >= score_IOMapped THEN
        technique = MEMORY_MAPPED_IO
    ELSE
        technique = IO_MAPPED_IO
    END IF
END IF

IF technique == MEMORY_MAPPED_IO THEN
    MAP device.registers INTO memory_address_space
    USE standard LOAD / STORE instructions
ELSE
    ASSIGN device.registers TO isolated_IO_address_space
    USE dedicated IN / OUT instructions
END IF

RETURN technique
END Select_IO_Technique

```

Result:

The algorithm first checks whether the request is bandwidth-heavy and large, favouring memory-mapped I/O since it can use the full instruction set, addressing modes, and DMA/cache paths. If instead the request has a strict latency bound, I/O-mapped I/O is chosen because dedicated IN/OUT instructions and an isolated address space give more predictable, faster access. When neither condition clearly dominates, a weighted score combining available bandwidth and current latency breaks the tie. This allows the system to adapt its I/O technique per-request instead of using a single fixed scheme.

Question 4

Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.

Aim:

To design a centralized bus arbitration algorithm that collects bus requests from PCIe, USB, and Thunderbolt devices, grants the shared bus to the highest-priority requester, and safely reclaims the bus after transfer or timeout.

Pseudocode:

```

STRUCTURE BusRequest
    device_id
    bus_type      // PCIe | USB | Thunderbolt
    priority      // derived from bus_type and QoS class
END STRUCTURE

DECLARE RequestQueue[]
DECLARE bus_status  = FREE
DECLARE current_owner = NULL

BEGIN Bus_Arbiter_Cycle()
    LOOP FOREVER

```

```

FOR EACH device IN {PCIe_devices, USB_devices, Thunderbolt_devices} DO
    IF device.BR (bus_request_line) == ACTIVE THEN
        priority = ASSIGN_PRIORITY(device.bus_type, device.QoS_class)
        INSERT BusRequest(device.id, device.bus_type, priority) INTO RequestQueue
    END IF
END FOR

IF RequestQueue NOT EMPTY AND bus_status == FREE THEN
    SORT RequestQueue BY priority DESCENDING
    // priority order: Thunderbolt (PCIe tunnel/display) > PCIe > USB

    winner = REMOVE_FIRST(RequestQueue)

    bus_status      = BUSY
    current_owner = winner.device_id
    SEND BG (bus_grant) TO winner.device_id

    WAIT UNTIL winner.device.BUSY_line == INACTIVE
        OR TIMEOUT(winner.max_transfer_time)

    IF TIMEOUT_OCCURRED THEN
        FORCE_RELEASE(winner.device_id)
        LOG("Bus timeout - device forcibly disconnected")
    END IF

    bus_status      = FREE
    current_owner = NULL
    CLEAR winner.BR line
END IF
END LOOP
END Bus_Arbiter_Cycle

FUNCTION ASSIGN_PRIORITY(bus_type, QoS_class)
    SWITCH bus_type
        CASE THUNDERBOLT: base_priority = 3    // supports PCIe tunnelling + display
        CASE PCIe:       base_priority = 2
        CASE USB:        base_priority = 1
    END SWITCH
    RETURN base_priority + QoS_class_bonus(QoS_class)
END FUNCTION

```

Result:

The arbiter continuously scans all connected devices for active bus-request lines, tags each with a priority derived from its interface type (Thunderbolt highest due to PCIe tunnelling and display data, then PCIe, then USB) and its QoS class, and queues them. When the bus is free, the highest-priority request is granted the bus; a timeout guard prevents a faulty or slow device from monopolizing the bus indefinitely. Once the transfer finishes (or times out), the bus is released and the cycle repeats, ensuring fair, priority-aware sharing of a single physical bus among heterogeneous interfaces.

Question 5

Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Aim:

To design a hybrid I/O manager that classifies devices by their data rate and services low-speed devices through simple polling while offloading high-speed device transfers to DMA, using interrupts only to signal DMA completion.

Pseudocode:

```

STRUCTURE Device
    id
    speed_class    // LOW_SPEED or HIGH_SPEED

```

```

    data_rate          // bytes/sec
END STRUCTURE

CONSTANT SPEED_THRESHOLD = 1,000,000    // 1 MB/s

BEGIN Hybrid_IO_Manager()
    FOR EACH device IN connected_devices DO
        IF device.data_rate < SPEED_THRESHOLD THEN
            device.speed_class = LOW_SPEED
        ELSE
            device.speed_class = HIGH_SPEED
        END IF
    END FOR

    LOOP FOREVER
        // ---- Low-speed devices: handled by POLLING ----
        FOR EACH device IN low_speed_devices DO
            status = READ device.status_register
            IF status == DATA_READY THEN
                data = READ device.data_register
                STORE data INTO buffer[device.id]
                CLEAR device.status_register
            END IF
        END FOR

        // ---- High-speed devices: handled by DMA + INTERRUPTS ----
        FOR EACH device IN high_speed_devices DO
            IF device.needs_transfer == TRUE AND device.DMA_channel == FREE THEN
                CONFIGURE DMA_controller(
                    source = device.port,
                    destination = memory_buffer[device.id],
                    length = device.pending_length,
                    interrupt_on_completion = TRUE
                )
                START DMA_controller
                device.DMA_channel = BUSY
            END IF
        END FOR

        SLEEP(poll_interval)    // brief delay before next polling round
    END LOOP
END Hybrid_IO_Manager

ON INTERRUPT DMA_COMPLETE(device_id) DO
    SAVE CPU.context
    MARK transfer_complete[device_id] = TRUE
    RELEASE DMA_channel[device_id]
    NOTIFY process_waiting_on(device_id)
    RESTORE CPU.context
    RETURN FROM INTERRUPT
END ON

```

Result:

Devices are first classified as LOW_SPEED or HIGH_SPEED based on a throughput threshold. In every cycle, the manager polls low-speed devices directly since the CPU overhead of polling a slow keyboard or sensor is negligible and avoids unnecessary interrupt overhead. For high-speed devices such as disks or network adapters, the manager instead configures and starts a DMA transfer, freeing the CPU immediately; completion is signalled only once via an interrupt. This hybrid approach combines the simplicity of polling for infrequent, slow events with the efficiency of interrupt-driven DMA for high-throughput transfers, balancing CPU overhead against responsiveness.

Code of Conduct:

*I **Deepika Shree. S (192521264)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____